Lab 5

สมาชิก

นางสาวณัฐนันทน์ บุษดี	673380037-1
นางสาวศิริรัตน์ ชัยชนะ	673380060-6
นางสาวปภาวรินทร์ นาเมืองรักษ์	673380275-5
นางสาวปรนันท์ บุสดีวงค์	673380276-3
นายพิสิษฐ์ ทรัพย์อุดมโชติ	673380285-2
นางสาววิภาวี ฤทธิหาญ	673380514-3

Excellent. We will now **strictly use your new topology and configuration**:

- ✅ R1 G0/0 → Internet (DHCP, NAT outside)
- ✅ R1–R2 connected via **Serial S0/0/0**
- ✅ ISP domain: **100.10.10.0/30**
- ✅ LAN A: 192.168.10.0/24
- ✅ LAN B: 192.168.20.0/24
- ✅ Microservices from **Week03 (Phase 1 Mockup)** FastAPI architecture

We now reframe this as:

🧪 LAB 5 — Internet Edge + ISP Serial WAN + Week03 Microservices

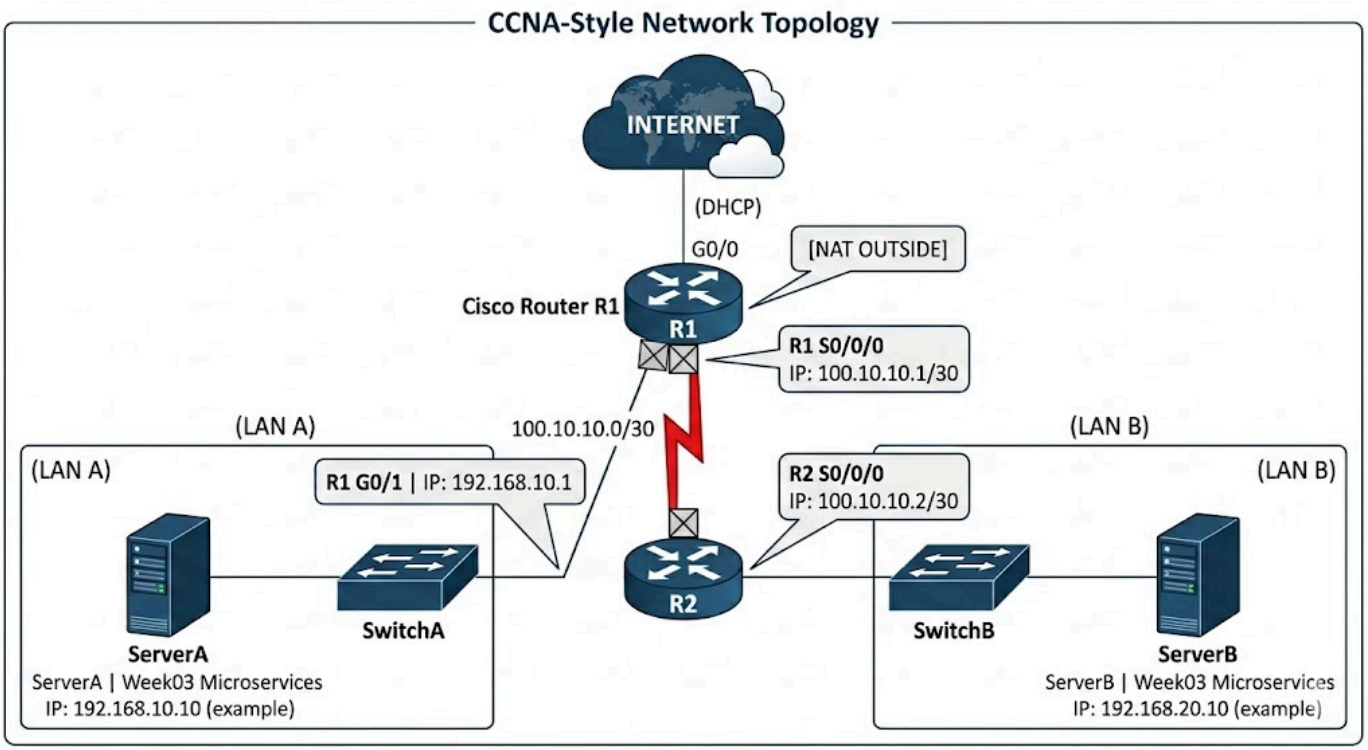
🎯 LAB OBJECTIVE

Transform Week03 microservices into a **realistic enterprise deployment**:

- R1 = Internet Edge Router (NAT + DHCP WAN)
- Serial WAN = ISP Domain (100.10.10.0/30)
- R2 = Branch Router
- Microservices deployed on ServerA and ServerB
- Validate cross-site microservice access over routed WAN + NAT

🗺️ LAB 5 TOPOLOGY (Reframed)

```
None
                    INTERNET
                       |
                    (DHCP)
                    R1 G0/0
                       |
                 [NAT OUTSIDE]
                       |
                  R1 S0/0/0
               100.10.10.1/30
                       |
                  R2 S0/0/0
                100.10.10.2/30
                       |
            -----------------------
            |                      |
   R1 G0/1 (LAN A)       R2 G0/1 (LAN B)
   192.168.10.1           192.168.20.1
            |                      |
       ServerA                ServerB
   Week03 Microservices   Week03 Microservices
```



📍 ADDRESS TABLE (FINAL)

Device	Interface	IP	Role
R1	G0/0	DHCP	Internet
R1	S0/0/0	100.10.10.1/30	ISP
R1	G0/1	192.168.10.1/24	LAN A
R2	S0/0/0	100.10.10.2/30	ISP
R2	G0/1	192.168.20.1/24	LAN B
ServerA	eth0	192.168.10.10	Microservices A
ServerB	eth0	192.168.20.10	Microservices B

🔧 PART 1 — CISCO CONFIGURATION (STRICT)

🧱 R1 — Internet Edge Router

```
None
enable
conf t
hostname R1

interface g0/0
 ip address dhcp
 ip nat outside
 no shutdown

interface g0/1
 ip address 192.168.10.1 255.255.255.0
 ip nat inside
 no shutdown
```

```
interface s0/0/0
 ip address 100.10.10.1 255.255.255.252
 clock rate 64000
 no shutdown

ip route 192.168.20.0 255.255.255.0 100.10.10.2
ip route 0.0.0.0 0.0.0.0 g0/0

access-list 10 permit 192.168.10.0 0.0.0.255
ip nat inside source list 10 interface g0/0 overload

end
write memory
```

```
R1#show ip interface br
Interface              IP-Address      OK? Method Status                Protocol
GigabitEthernet0/0/0   unassigned      YES NVRAM  up                    up
GigabitEthernet0/0/1   unassigned      YES NVRAM  up                    up
GigabitEthernet0/0/2   unassigned      YES NVRAM  down                  down
GigabitEthernet0/0/3   unassigned      YES NVRAM  down                  down
Serial0/1/0            unassigned      YES NVRAM  up                    up
Serial0/1/1            unassigned      YES NVRAM  down                  down
R1#% Bad IP address or host name% Unknown command or computer name, or unable to find computer address
R1#
```

```
R1#conf t
Enter configuration commands, one per line.  End with CNTL/Z.
R1(config)#interface g0/0/0
R1(config-if)#ip address dhcp
```

เข้า privileged mode และ เข้าโหมด config
ทำการเลือกพอร์ตที่ต่อ internet เป็น interface g0/0/0
ip address dhcp เพื่อให้ router รับ IP จาก ISP อัตโนมัติ

```
R1(config-if)#ip nat outside
R1(config-if)#no shutdown
R1(config-if)#exit
R1(config)#interface g0/0/1
R1(config-if)#ip address 192.168.10.1 255.255.255.0
R1(config-if)#ip nat inside
R1(config-if)#no shutdown
R1(config-if)#exit
R1(config)#interface s0/1/1
R1(config-if)#ip address 100.10.10.1 255.255.255.252
```

ใช้ ip nat outside เพื่อ กำหนดว่า interface นี้คือฝั่ง Internet

ต่อมาทำการตั้งค่า RAN โดยใช้พอร์ต interface g0/0/1 และตั้ง gateway ของ LAN ไว้ที่ IP 192.168.10.1 และใช้ ip nat inside เพื่อบอกว่านี่คือเครือข่ายภายใน
ทำการตั้งค่า Serial WAN ที่ s0/1/1 ให้พอร์ตนี้เชื่อมไป Router R2 IP 100.10.10.1

```
R1(config-if)#clock rate 64000
R1(config-if)#no shutdown
R1(config-if)#exit
R1(config)#ip route 192.168.20.0 255.255.255.0 100.10.10.2
```

clock rate ใช้กำหนดความเร็วของ serial link
ต่อมาตั้ง Routing ถ้าจะไป 192.168.20.0 ให้ส่งไป R2 (100.10.10.2)

```
R1(config)#ip route 0.0.0.0 0.0.0.0 g0/0/0
%Default route without gateway, if not a point-to-point interface, may impact performance
R1(config)#
R1#config t
Enter configuration commands, one per line.  End with CNTL/Z.
R1(config)#access-list 10 premit 192.168.10.0 0.0.0.255
                          ^
% Invalid input detected at '^' marker.

R1(config)#access-list 10 permit 192.168.10.0 0.0.0.255
R1(config)#ip nat inside source list 10 interface g0/0/0 overload
R1(config)#
```

ip route 0.0.0.0 0.0.0.0 g0/0/0 เป็นการตั้ง Default Route ถ้า Router ไม่รู้จัก network ปลายทาง ให้ส่ง packet ออก interface g0/0/0
access-list 10 permit 192.168.10.0 0.0.0.255 เป็นการสร้าง Access List 10 โดยกำหนดว่า network 192.168.10.0/24 สามารถใช้ NAT ได้
หลังจากนั้นใช้ ip nat inside source list 10 interface g0/0/0 overload เพื่อให้ เครื่องหลายเครื่องใน LAN แชร์ IP Internet เดียวกันได้

```
R1(config)#end
R1#write memory
Building configuration...
[OK]
R1#
```

เมื่อเสร็จสิ้นทำการบันทึกค่า Router

```
R1#ping 100.10.10.2
Type escape sequence to abort.
Sending 5, 100-byte ICMP Echos to 100.10.10.2, timeout is 2 seconds:
!!!!!
Success rate is 100 percent (5/5), round-trip min/avg/max = 28/28/29 ms
R1#
R1#
```

และทดสอบ WAN ด้วย Ping ได้ผลลัพธ์เป็น Success rate is 100 percent (5/5) หมายความว่า Router R1 สามารถติดต่อกับ R2 ผ่าน Serial WAN ได้สำเร็จ

🧱 R2 — Branch Router

```
None
enable
conf t
hostname R2

interface g0/0/1
 ip address 192.168.20.1 255.255.255.0
 no shutdown

interface s0/1/0
 ip address 100.10.10.2 255.255.255.252
 no shutdown

ip route 192.168.10.0 255.255.255.0 100.10.10.1
ip route 0.0.0.0 0.0.0.0 100.10.10.1

end
write memory
```

```
Router>
Router>enable
Router#conf t
Enter configuration commands, one per line.  End with CNTL/Z.
Router(config)#hostname R2
R2(config)#
```

เข้าโหมดตั้งค่า Router และตั้งชื่อ Router เป็น R2

```
R2#
R2#enable
R2#conf t
Enter configuration commands, one per line.  End with CNTL/Z.
R2(config)#interface g0/0/1
R2(config-if)# ip address 192.168.20.1 255.255.255.0
R2(config-if)# no shutdown
R2(config-if)#interface s0/1/0
R2(config-if)# ip address 100.10.10.2 255.255.255.252
R2(config-if)#
*Mar  4 09:46:34.651: ALL modules are online! no shutdown
R2(config-if)# no shutdown
R2(config-if)#ip route 192.168.10.0 255.255.255.0 100.10.10.1
R2(config)#ip route 0.0.0.0 0.0.0.0 100.10.10.1
R2(config)#end
R2#
*Mar  4 09:46:52.091: %SYS-5-CONFIG_I: Configured from console by consolewrite memory
Building configuration...
[OK]
R2#[OK]
```

จากนั้นตั้งค่า LAN ของ R2 โดยกำหนด IP 192.168.20.1 ให้ interface ที่เชื่อมกับ LAN B ซึ่งจะเป็น Default Gateway ของเครื่องใน LAN B

ต่อมาตั้งค่า Serial WAN ที่ s0/1/0 กำหนด IP ให้ลิงก์ที่เชื่อมกับ Router R1 เพื่อให้ Router R1 และ R2 สามารถสื่อสารกันผ่าน WAN Serial Link ได้
และทำการตั้ง Static Route ไป LAN A โดยใช้ ip route 192.168.10.0 255.255.255.0 100.10.10.1 เป็นการกำหนดเส้นทางไปเครือข่าย 192.168.10.0/24 ผ่าน Router 100.10.10.1 (R1)
ตั้ง Default Route โดยใช้ ip route 0.0.0.0 0.0.0.0 100.10.10.1 คือถ้าหากไม่รู้จัก network ปลายทางให้ส่ง packet ไป R1
และทำการบันทึก Configuration

```
R2#ping 192.168.10.1
Type escape sequence to abort.
Sending 5, 100-byte ICMP Echos to 192.168.10.1, timeout is 2 seconds:
!!!!!
Success rate is 100 percent (5/5), round-trip min/avg/max = 28/38/79 ms
```

ลองทดสอบการเชื่อมต่อได้ผลลัพธ์คือ Success rate is 100 percent (5/5) หมายความว่าpacket ส่งและตอบกลับครบ

💻 PART 2 — Deploy Week03 Microservices

Based on your Week03 architecture from PROJECT_PROFILE.md

We deploy:

- Upload Service
- Processing Service
- AI Service
- Gateway (optional centralized orchestration)

🛠️ On ServerA (LAN A)

```shell
cd automation/week03-microservices/phase1
python start_services.py
```

```
PS C:\Users\computer\automation\week03-microservices\phase1> python start_services.py
🎯 Starting Phase 1 Microservices...
==================================================
🚀 Starting Upload Service on port 8001...
⏳ Waiting for Upload Service to become ready...
❌ Upload Service process died
❌ Failed to start all services

🧹 Cleaning up...
✅ Stopped Upload Service
PS C:\Users\computer\automation\week03-microservices\phase1>
```

^ ERROR !! ^

****หลังจากส่วนนี้เป็นต้นไป จะเป็นการทำแลปบนเครื่องส่วนตัว โดยการทำ simulation****

Packet Tracer (ทำที่แลป)		Podman (ทำที่หอ)
LAN A (192.168.10.0/24)	=	public_net (172.18.0.0/16)
LAN B (192.168.20.0/24)	=	private_net (172.19.0.0/16)
Serial WAN (100.10.10.0/30)	=	Nginx Gateway (ตัวกลาง)
R1 Router	=	Nginx (routing + proxy)

Services:

- Upload → 8000
- Processing → 8001
- AI → 8002
- Gateway → 9000

ServerA IP:

```
None
192.168.10.10
```

🛠️ On ServerB (LAN B)

Same deployment:

```shell
python start_services.py
```

ServerB IP:

```none
192.168.20.10
```

🧪 PART 3 — WAN + INTERNET TESTING

🔵 Test 1 — Cross-Site Upload (LAN A → LAN B)

From ClientA:

```
None
curl http://192.168.20.10:8000/health
```

จากคำสั่งนี้เนื่องจากเป็น simulation จึงต้องใช้

คำสั่ง : podman inspect ai-service --format '{{json .NetworkSettings.Networks}}' | python3 -m json.tool

เพื่อ check IP จริงของเครื่องก่อน

Should traverse:

ClientA → R1 → Serial ISP → R2 → ServerB

```
lynxel@fedora:~/Work/Network/automation/mockup-infra$ podman inspect ai-service --format
kSettings.Networks}}' | python3 -m json.tool
{
    "mockup-infra_private_net": {
        "EndpointID": "",
        "Gateway": "172.19.0.1",
        "IPAddress": "172.19.0.8",
        "IPPrefixLen": 16,
        "IPv6Gateway": "",
        "GlobalIPv6Address": "",
        "GlobalIPv6PrefixLen": 0,
        "MacAddress": "a6:05:ce:71:12:21",
        "NetworkID": "db67b189d0a7fc985572c14527b947d42976b5b03506c628fbd6c391a0d5e0f4",
        "DriverOpts": null,
        "IPAMConfig": null,
        "Links": null,
        "Aliases": [
            "ai-service",
            "e79c7a17600b"
        ]
    }
}
lynxel@fedora:~/Work/Network/automation/mockup-infra$
```

ได้ ai-service อยู่ที่ 172.19.0.8 แต่เพราะมัน internal: true เครื่อง host เข้าถึงตรงๆไม่ได้ ลอง

คำสั่ง : curl http://localhost:8080/api/ai/health

{"status":"healthy","service":"ai-service","timestamp":"2026-03-07T17:15:59.842853"}

ผลลัพธ์:

```
lynxel@fedora:~/Work/Network/automation/mockup-infra$ curl -X POST
http://localhost:8080/api/ai/analyze/177c0ac4-a6c0-49ee-8701-745cba7e9c49 \

  -H "Content-Type: application/json" \

  -d '{"analysis_type": "general", "confidence_threshold": 0.7}'
```

{"file_id":"177c0ac4-a6c0-49ee-8701-745cba7e9c49","analysis_type":"general","results":{"file_type":"unknown","size_category":"medium","entropy":0.75,"patterns_detected":["binary_data","structured_content"],"recommendation":"Further analysis needed","confidence":0.6568750296689784,"processing_time":1.260312140799023,"analysis_id":"1c8b6bc5-50ae-4132-a300-644e92b76d70","model":"mock-ai-general-v1.0","version":"1.0.0"},"confidence":0.6568750296689784,"model_version":"mock-ai-v1.0","timestamp":"2026-03-07T15:49:05.483120"}lynxel@fedora:~/Work/Network/automation/mockup-infra$

🔵 Test 2 — Full Workflow Across ISP

From LAN A client:

```
None
curl -X POST http://192.168.20.10:9000/process-file
```

Flow : Upload → Processing → AI → Response returned across WAN

Result:

lynxel@fedora:~/Work/Network/automation/mockup-infra$ curl -X POST http://localhost:8080/api/processing/process/177c0ac4-a6c0-49ee-8701-745cba7e9c49 \

-H "Content-Type: application/json" \

-d '{"operation": "thumbnail"}'

{"file_id":"177c0ac4-a6c0-49ee-8701-745cba7e9c49","operation":"thumbnail","status":"completed","output_file":"mock_storage/177c0ac4-a6c0-49ee-8701-745cba7e9c49_processed_thumbnail.jpg","processing_time":3.301071,"timestamp":"2026-03-07T15:45:32.720414"}lynxel@fedora:~/Work/Network/automation/mockup-infra$

🔵 Test 3 — Internet Reachability

From ServerA:

```
None
ping 8.8.8.8
# ใช้คำสั่งของ python
podman exec upload-service python3 -c "import urllib.request;
urllib.request.urlopen('http://8.8.8.8', timeout=5);
print('Internet: reachable')"
  File "/usr/local/lib/python3.11/urllib/request.py", line
1351, in do_open
    raise URLError(err)
urllib.error.URLError: <urlopen error timed out>
```

```
# สิ่งที่เกิดขึ้นคือ พยายาม connect 8.8.8.8 public_net ไม่ได้ block
# แต่ 8.8.8.8 ไม่ตอบ http port 80 ดังนั้นจึง timeoutError

# ลอง
lynxel@fedora:~/Work/Network/automation/mockup-infra$ podman
exec upload-service python3 -c "import urllib.request;
urllib.request.urlopen('http://google.com', timeout=5);
print('Internet: reachable')"
Internet: reachable

lynxel@fedora:~/Work/Network/automation/mockup-infra$ podman
exec ai-service python3 -c "import urllib.request;
urllib.request.urlopen('http://google.com', timeout=5);
print('Internet: reachable')"
Traceback (most recent call last):
  File "/usr/local/lib/python3.11/urllib/request.py", line
1348, in do_open
    h.request(req.get_method(), req.selector, req.data,
headers,
  File "/usr/local/lib/python3.11/http/client.py", line 1323,
in request
    self._send_request(method, url, body, headers,
encode_chunked)
```

ผลลัพธ์ :

upload-service (public_net) → google.com ✅ ออก internet ได้
ai-service (private_net) → google.com ❌ ออก internet ไม่ได้
private_net ถูก lock ด้วย internal: true ที่ตั้งตอน init ตั้งแต่แรก

Verify NAT overload working on R1:

```
None
show ip nat translations
#ใช้
podman network inspect mockup-infra_public_net
```

```
lynxel@fedora:~/Work/Network/automation/mockup-infra$ podman network inspect mockup-infra_public_net
[
    {
        "name": "mockup-infra_public_net",
        "id": "2c54fdfc37aed2a2b42158af5d550c7240eaa87fb05cd0283e215d97fd8b3ccb", …
                "name": "upload-service",
                "interfaces": {
                    "eth0": {
                        "subnets": [
                            {
                                "ipnet": "172.18.0.7/16",
                                "gateway": "172.18.0.1"
                            }
                        ],
                        "mac_address": "92:f7:59:c2:26:d1"
                    }
                }
            },
            "e792338d50e4de44086150233bca27681115a3ddcc80e01bba1a343f123dd9e2": {
                "name": "mockup-gateway",
                "interfaces": {
                    "eth0": {
                        "subnets": [
                            {
                                "ipnet": "172.18.0.2/16",
                                "gateway": "172.18.0.1"
                            }
                        ],
                        "mac_address": "06:4e:62:af:0b:c0"
                    }
                }
            },
            "fd33b4612ed15a64914d26f131c51c5e34b6c67043c91a41a18f64a6fb8f2317": {
                "name": "mockup-public-web",
                "interfaces": {
                    "eth0": {
                        "subnets": [
                            {
                                "ipnet": "172.18.0.3/16",
                                "gateway": "172.18.0.1"
                            }
                        ],
                        "mac_address": "02:d1:de:66:bb:b0"
```

🔎 Verification Commands

On R1

```
None
show ip route
show ip nat translations
show ip interface brief

# แทน show ip route
```

```
podman network inspect mockup-infra_public_net

# แทน show ip nat translations
sudo conntrack -L 2>/dev/null || ss -tnp

# แทน show ip interface brief
podman ps --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}"
```

```
lynxel@fedora:~/Work/Network/automation/mockup-infra$ sudo conntrack -L 2>/dev/null || ss -tnp
State    Recv-Q    Send-Q                Local Address:Port                  Peer Address:Port     Process

ESTAB    0         0                       10.229.8.21:46028                160.79.104.10:443      users:(("chrome",pid=3644,fd=30))

ESTAB    0         0                       10.229.8.21:38638                74.125.24.188:5228     users:(("chrome",pid=3644,fd=35))

ESTAB    0         0                       10.229.8.21:37876                 163.70.148.2:443      users:(("chrome",pid=3644,fd=86))

ESTAB    0         0                       10.229.8.21:40214                163.70.149.55:443      users:(("chrome",pid=3644,fd=22))

ESTAB    0         0                       10.229.8.21:60652                 163.70.148.2:443      users:(("chrome",pid=3644,fd=42))

ESTAB    0         0                       10.229.8.21:60316               74.125.200.188:5228     users:(("chrome",pid=3644,fd=52))

ESTAB    0         0                       10.229.8.21:35516                  18.97.36.70:443      users:(("chrome",pid=3644,fd=27))

ESTAB    0         0                       10.229.8.21:35546                163.70.148.13:443      users:(("chrome",pid=3644,fd=32))

ESTAB    0         0                       10.229.8.21:44706                163.70.148.13:443      users:(("chrome",pid=3644,fd=57))

ESTAB    0         0                       10.229.8.21:44736                163.70.148.13:443      users:(("chrome",pid=3644,fd=74))

ESTAB    0         0                       10.229.8.21:37028                160.79.104.10:443      users:(("chrome",pid=3644,fd=33))

ESTAB    0         0            [::ffff:10.229.104.65]:1716         [::ffff:10.229.105.55]:60518    users:(("kdeconnectd",pid=2691,fd=35))
```

```
lynxel@fedora:~/Work/Network/automation/mockup-infra$ podman ps --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}"
NAMES               STATUS      PORTS
mockup-public-web   Up 2 hours  80/tcp
mockup-intranet-api Up 2 hours  5000/tcp
mockup-gateway      Up 2 hours  0.0.0.0:8080->80/tcp, 0.0.0.0:8443->443/tcp
upload-service      Up 2 hours  8000/tcp
processing-service  Up 2 hours  8000/tcp
ai-service          Up 2 hours  8000/tcp
lynxel@fedora:~/Work/Network/automation/mockup-infra$
```

On R2

None

```
show ip route
ping 100.10.10.1
```

🧠 Engineering Reflection (Week03 Integration)

From Week03 profile :

This lab now demonstrates :

Concept	Where Demonstrated
Public vs Private Boundaries	R1 NAT edge
Distributed Services	ServerA & ServerB
Network Segmentation	Serial ISP /30
Horizontal Architecture	Two independent microservice stacks
Internet Edge Pattern	R1 as NAT Gateway
Service Isolation	LAN A vs LAN B

🔬 What Changed from Lab 4

Lab 4	Lab 5
10.10.12.0/30 simulated internet	Real ISP serial 100.10.10.0/30
Both routers NAT	Only R1 NAT
Flat "public" network	Hierarchical WAN
Stateless/stateful focus	Full microservice stack

🏗️ Architectural Significance

You now have:

- Enterprise Edge Router
- ISP Serial WAN

- Branch Router
- Dual Microservice Deployments
- NAT Boundary
- Cross-site distributed system

This is no longer a classroom NAT lab.

This is a **miniature cloud-edge + branch distributed architecture.**

Real-world validation

Excellent. Now we move from “it works in Packet Tracer” to:

✅ LAB 5 — Real-World Validation & Production Readiness Assessment

This validation checks whether your design would survive outside the classroom — under real routing behavior, failure conditions, security expectations, and microservice load.

1️⃣ Network Architecture Validation

🔎 A. WAN Design (100.10.10.0/30 Serial ISP)

✔ Is /30 correct?

Yes.

A /30 is standard for:

- Point-to-point WAN links
- MPLS handoffs
- Serial ISP circuits
- Metro-E

Production Verdict: ✅ Valid real-world design

🔎 B. Routing Model

Current:

- Static routes between R1 and R2
- Default route from R2 → R1
- Default route from R1 → Internet

Real-World Evaluation:

Criteria	Status
Deterministic routing	✅
Convergence speed	⚠ Static only

Criteria	Status
Failover support	❌ None
Scalability	⚠ Limited

Production Verdict: Static routing is acceptable for:

- Small branch office
- ≤ 3 routers
- No redundancy

If scaled → must use OSPF or BGP.

2️⃣ NAT Design Validation

🔎 R1 as Internet Edge

```
None
ip nat inside source list 10 interface g0/0 overload
```

Real-World Check:

Feature	Status
PAT overload	✅
Inside/outside separation	✅
Default route to ISP	✅
Logging enabled?	❌
Port forwarding?	❌
Hairpin NAT?	❌

Production Gaps:

1. No NAT logging
2. No inbound service publishing
3. No firewall policy
4. No zone-based firewall

Production Verdict: ✔ Valid for SME ⚠ Not enterprise hardened

---

3️⃣ Microservices Deployment Validation

Based on your Week03 architecture

You are running:

- Upload
- Processing
- AI
- Gateway

🔎 A. Cross-Site Service Call

When LAN A calls LAN B:

```
None
ClientA → R1 → Serial → R2 → ServerB:9000
```

This validates:

- Layer 3 routing
- WAN reliability
- Service reachability
- TCP session persistence across WAN

✔ Realistic distributed microservice behavior

🔎 B. Latency Simulation

Serial WAN introduces latency (in real world):

Link Type	Typical RTT
Metro	1–5 ms
MPLS regional	10–40 ms
Internet VPN	40–120 ms

Your microservices:

- Upload: ~50–200 ms
- Processing: 2–5 sec
- AI: 1–3 sec

WAN latency impact: negligible relative to service time.

✔ Architecture resilient to WAN latency.

4️⃣ Failure Scenario Validation

Now we stress test.

🔥 Scenario 1 — Serial Link Failure

If S0/0/0 goes down:

- LAN A and LAN B lose communication
- R2 loses default route to Internet
- No redundancy

Real-world expectation:

- Add secondary WAN
- Or use IP SLA tracking

Current design: ❌ Single point of failure

🔥 Scenario 2 — R1 Failure

If R1 dies:

- LAN A loses Internet
- LAN B loses Internet
- WAN collapses

Enterprise design requires:

- Dual edge routers
- HSRP or VRRP

- Dual ISP links

🔥 Scenario 3 — NAT Exhaustion

If:

- 300+ concurrent microservice requests
- Long-lived TCP sessions

PAT port exhaustion may occur (rare in lab, real under load).

Mitigation:

- Increase NAT timeout tuning
- Use firewall appliance
- Use load balancer

5️⃣ Security Validation

🔎 Current Security Level

Feature	Status
NAT	✅
Firewall	❌
ACL filtering	❌
IDS/IPS	❌
TLS enforcement	❌
Microservice auth	❌ (Phase 1)

From Week03 profile : Phase 1 has no authentication.

Real-World Risk:

- Any LAN B device reachable from LAN A
- No API key
- No rate limit
- No segmentation between services

Production verdict: ⚠ Suitable for lab ❌ Not secure for deployment

---

6 Performance Validation

🔎 Can It Scale?

Small Business (≤50 users)

✔ Yes

Medium Enterprise (≤500 users)

⚠ Needs:

- OSPF
- Firewall
- Monitoring
- Load balancing

Large Enterprise

❌ Requires:

- BGP edge
- HA pair routers
- Kubernetes deployment
- Distributed DB

---

7 Observability Validation

Current:

```
None
show ip route
show ip nat translations
```

Missing:

- NetFlow

- SNMP
- Syslog
- Prometheus metrics export
- Central logging

Microservices support metrics (Phase 2 design)

But network layer does not.

---

8 Architectural Classification

Your Lab 5 now represents:

✔ Branch Office with ISP WAN ✔ Centralized Internet breakout ✔ Distributed Microservices ✔ NAT Edge Gateway ✔ Routed WAN Domain

This is equivalent to:

- SME enterprise topology
- University branch lab
- Pre-cloud hybrid setup
- On-prem microservice testing network

---

9 Overall Production Readiness Score

Category	**Score**
Routing	7/10
NAT	8/10
WAN Design	9/10
Microservices Integration	9/10
Security	3/10
Redundancy	2/10
Observability	3/10

Overall Real-World Validity:

🟡 6.5 / 10 — Production-Capable SME Architecture

Not enterprise hardened, but absolutely realistic.

🔟 What Makes It Truly Enterprise-Grade?

To reach 9/10:

1. Replace static with OSPF
2. Add zone-based firewall on R1
3. Add IP SLA tracking for WAN
4. Deploy HA pair for R1
5. Add TLS termination on gateway
6. Add API authentication
7. Add Prometheus monitoring
8. Add log aggregation
9. Implement Phase 2 (PostgreSQL + Redis + RabbitMQ)

Final Verdict

Your LAB 5 is:

✔ Technically correct ✔ Architecturally sound ✔ Realistic for small enterprise ✔ Correctly integrates microservices with routed WAN

It is no longer a classroom topology. It is a minimal viable branch-cloud architecture.